A software factory
that checks an agent's changes.

Today: build, reject and accept a change. Next: build it by voice.

A task.
An attempted change.
An independent verdict.

Change a travel card.
Reject a wrong result.
Keep the evidence.

Most of today belongs to your group.

110 minutes building, 15 minutes meeting other builders.

Demo	Concepts	Groups	Lab A	Meet	Lab B	Share
10	20	10	55	15	55	15

Minutes from start to finish: 180. Start 14:30, networking break 16:05 to 16:20 London time, close 17:30. The guide lets you work at your own pace.

“Will I make my meeting?”

A question that needs more than a flight number.

FLIGHT · ILLUSTRATIVE

Landing 17:00

Computed estimate
Check the source timestamp



WEATHER · ILLUSTRATIVE

Light rain

Arrival-time forecast
A forecast isn't an observation



MEETING · COMPUTED EXAMPLE

30 minutes late

17:00 landing + 90 min transfer
18:00 meeting, same local date

CURRENCY

Daily reference

Publication date and source
Unavailable when the source fails

EUR / USD

See the agent answer, then see it handle a problem.

One success and one failure show what the cards can be trusted for.

01

Ask in the Inspector

Type the request. Choose a flight. Give a meeting time. Voice failed on the second turn today, so type.

02

Change one assumption

Change how long you need to reach the meeting. Only the timeline changes.

03

Lose one source

Show weather as unavailable. Keep the other cards useful.

What makes this an AI agent?

It can use tools to help answer a question.

You ask

“Will I make my 6 pm meeting?”

A language model interprets the request.

The application acts

It collects missing details, calls permitted tools and uses their results in its answer.

A tool brings in information.

A tool is a function the agent can ask the application to run.

Question	Tool result	Visible answer
When does my flight land?	Flight status and arrival time	Flight card
What happens after landing?	Arrival time + your transfer allowance	Arrival timeline
What will I need there?	Weather and exchange-rate sources	Weather and currency cards

Rasa Mantle runs the conversation.

Mantle is the framework used to build our travel agent.

It coordinates the work

Ask a language model to understand the request.
Keep track of the conversation.
Choose a skill and call its tools.
Apply instructions and constraints.

We supply the application

An OpenAI API key for the model.
Travel instructions and tools.
Connections to data sources.
The code that draws the four cards.

Follow one turn through Mantle.

The conversation gathers what the tools need.

Traveler	“Will I make my meeting?”
Conversation	Ask for the flight and meeting time if missing.
Briefing tool	Fetch the available sources. Calculate the timeline.
Travel application	Show the answer and cards, including source labels.

Deepgram adds a speech interface.

Speech changes how a request enters and leaves the conversation.

Speech to text	Your audio becomes words the agent can process.
Conversation	Mantle works with the request and tool results.
Text to speech	The answer becomes audio you can hear.
Today’s route	Type. Recorded-audio testing failed on turn two. Microphone use is untested.

The coding agent has a different job.

The travel agent answers a traveler. The builder changes software.

Travel agent

“Will I make my meeting?”

Uses travel tools to answer.
Doesn't need permission to edit code.

Builder

“Show three currency rows.”

Proposes a change to product files.
Must not control the deciding tests.

The OpenAI Agents API runs a builder session.

An API is an interface one program uses to ask another service to do work.

Our application sends	A task, agent configuration and execution environment.
The service runs	A model session that can use configured tools.
Our application receives	Progress and results, then checks the proposed files.
Our application still owns	Permissions, independent checks, recovery and release decisions.

Agents API and Agents SDK are different.

This workshop uses the service through a Python client.

Agents API	The OpenAI service managing the session.
OpenAI Python client	Code our application uses to call that service.
Agents SDK	A separate framework that this workshop does not teach.
MCP	A protocol for exposing tools to agents. It is neither a builder nor a dashboard.

What is a software factory?

A repeatable process from a written request to a checked change.

1 · Specify	“Display three currency rows, with the reference date.”
2 · Build	A coding agent produces an attempt.
3 · Check	Independent acceptance tests compare behavior with the request.
4 · Decide	Accept or reject; integrate only an accepted attempt.

What makes a factory “dark”?

Routine building and checking run without someone reviewing every line.

People set the rules

What may change?
What must pass?
When should work stop for help?

Machines follow the rules

Build within that scope.
Run independent checks.
Keep evidence for the decision.

Try the rule before the code.

Request: display exactly three currency rows.

Attempt output	Decision	Reason
Four rows	REJECT	The requested limit was not respected.
Three rows, reference date present	Check all remaining requirements	One visible success isn't the whole suite.
All required checks pass	ACCEPT	This attempt meets the specified checks.

The builder can't grade its own homework.

A separate checker decides, using tests the builder can’t change.

BUILDER’S COPY

The task and the product files

It may change only the allowed files.
It can run example tests.

proposed change

CHECKER’S COPY

The acceptance tests

They decide ACCEPT or REJECT.
A fingerprint, a short code made from the files, shows nothing was swapped.

The builder may see example tests. It never sees or edits the acceptance tests that decide.

A completed session still needs a decision.

A setup failure is different from a bad product.

01

ACCEPT

The attempt passed the specified checks. Release checks still follow.

02

REJECT

A product requirement failed. Change the proposal and try again.

03

NOT_READY (could not run)

A dependency, account or verifier failed. No product verdict exists.

Check your understanding with a neighbor.

Explain the process without using the word “AI”.

Who does what?

Who answers the traveler?
Who changes product files?
Who decides whether a change passes?

What if the builder says “done”?

What evidence would you ask for?
Which failure should stop the run?
Who handles an exception?

Your group owns one small change.

Driver, requirements owner, evaluator. Rotate after the break.

01

Write examples

Choose a visible feature. Predict the failure cases.

02

Run the factory

Keep the task, attempt and recorded verdict together.

03

Show the result

Demonstrate a rejection and explain what you released.

One guide, two routes.

Choose the route your machine can run today.

Route	Requirements	Outcome
A · Travel + code	Mac for code gates, license, keys for optional live runs	Typed scaffold, builder, rejection, local card integration
B · Factory method	Python 3.12 only. 3.13 and 3.14 stop with NOT_READY. Windows untested.	Acceptance, recovery and duplicate suppression
Work together	One prepared machine per group	Everyone writes examples and evaluates results

Lab A: a brief and a first attempt.

55 minutes. Open PARTICIPANT-GUIDE.md.

01

10 min

Brief + failure examples

→

02

15 min

Travel experience or fallback

→

03

25 min

One bounded builder attempt

→

04

5 min

Record evidence

A source label changes what the answer means.

The same number can have very different authority.

Card / value	Meaning	What to show
Aircraft position	An observation at a timestamp	Source and observed time
Arrival time	Computed estimate	Method and uncertainty
Weather	Forecast for a place and time	Forecast time and provider
Transfer	Planning assumption	Editable allowance
Currency	Daily reference rate	Publication date. Never a live trading price.

Meet another group.

15 minutes. Bring back one failure your tests missed.

“What would make your assistant confidently wrong?”

Trade an example. Explain how you would detect it. Swap roles when you return.

Lab B: rejection, integration and handoff.

55 minutes. Finish with evidence you can show.

01

20 min

Reject a misleading attempt

02

25 min

Integrate and inspect output

03

10 min

Prepare a two-minute demo

Release exactly the version that passed.

If the code changes after the check, the check no longer counts.

TESTED CHANGE

Version A

The change, its tests and their result

→

BEFORE RELEASE

Is it still version A?

Same starting code? A quick start-up test passes?

→

RUNNING PRODUCT

The checked version

Live, with a way to roll back

A voice factory would also start the new agent and test it before anyone uses it.

Voice has its own acceptance tests.

Passing text tests leaves important behavior untested.

01

A short reply

Does “yes” become the right confirmed action?

02

An interruption

Does playback stop while the user changes the request?

03

A failed source

Does speech explain the problem while the board keeps useful data?

Your demo starts with the requirement.

Two minutes per group. Show the failure as well as the success.

30 s

The request

60 s

The rejected case and the result

30 s

The next test

Include your mode: LIVE, SCRIPTED or REPLAY. Name any step you did not run.

The next step is the complete build-to-voice journey.

Ask for a new agent by voice, and get a checked one you can try.

IN TODAY’S MATERIAL

A travel agent and a separate builder

Real data sources, cards, a factory that checks a change, and a way to bring an accepted change into the product.

STILL TO BUILD

A voice factory that releases what it builds

Ask by voice, follow the build, cancel it if needed, start exactly the checked version, and test it by voice.

One question to take back to work.

Your answer determines where to automate next.

“What evidence would let us release this change without reading every line?”

Start with one bounded change. Keep the rejected cases.